Reclaiming Closed Service Robots: A Hypothesis-Driven Methodology for Non-Destructive Protocol Reconstruction

Anonymous Authors
Affiliation withheld for double-blind review

***Abstract*—Commercial service robots are increasingly sold as closed Android–ROS systems: no public SDK, no message schemas, and a vendor cloud that the owner cannot replace. A laboratory that owns such a robot cannot modify its reception behaviour, and cannot even establish which commands the machine accepts. We present a non-destructive methodology for recovering that command surface without vendor cooperation, structured as four falsifiable hypotheses about where the protocol is observable and how to verify it, tested through a seven-phase pipeline that triangulates network mapping, `rosapi` introspection, static analysis of the vendor application, and passive broker observation. We validate the methodology on a commercially available reception robot, recovering enough of its command surface to rebuild teleoperation, autonomous navigation, speech output and visitor interaction as an edge platform that runs without the vendor cloud. Our main empirical finding concerns verification rather than discovery. A transport-level acknowledgment says nothing about whether the machine moved. In 17 logged field commands, what separated success from failure was the localization state the robot reported: all ten failures occurred while it reported itself unlocalized, and all seven successes while it reported navigating. How the goal was addressed made no difference at all. A controlled bench comparison shows the same asymmetry between an unprepared and a prepared command path. We report field indicators with confidence intervals, a capability comparison against the vendor application, and the conditions under which each hypothesis should transfer to other closed platforms.**

***Index Terms*—Service robotics, reverse engineering, vendor lock-in, ROSBRIDGE, edge computing**

I. Introduction

A laboratory that buys a commercial service robot buys a sealed appliance. The vendor ships an Android application, a cloud account, and a support contract; it does not ship the message schemas that would let the owner send the robot anywhere the application does not already go. For research groups this is a hard limit: the platform in Fig. 1 navigates, speaks and greets visitors perfectly well, but only in the ways its manufacturer anticipated.

Why this is hard. Three obstacles compound, and none of them is peculiar to this vendor. First, the robot is not one computer but two: an Android head and a ROS chassis, joined by an undocumented internal link. No single point of observation covers the whole command surface, and each one is blind in a different place (Section IV). Second, there is no white-box route. Shells on the chassis are closed, and repeated authentication attempts lock the account, so we cannot simply read the running system. Third, and least obvious, the software layer that accepts a message is not the layer that turns the wheels. A command can be accepted, acknowledged and then quietly dropped. Treat the acknowledgment as proof and the result is a protocol description that is confidently wrong. This is the reason naive fuzzing of the exposed interfaces gets nowhere on such a platform.

Fig. 1. The study platform running our reconstructed visitor interface on its own touchscreen. The head is an Android computer; the mobile base underneath is a separate Linux/ROS machine. Neither exposes a documented interface to its owner. Identifying marks are blurred.

Research question. *Can a non-destructive methodology recover the command surface of a closed Android–ROS robot without documentation, and what evidence is required before a recovered command may be considered validated?*

We formulate four hypotheses, stated here and resolved in Section VI:

- **H1 (application-as-specification).** Static analysis of the vendor's own Android application yields the wire protocol the chassis actually speaks.
- **H2 (broker-as-command-channel).** The exposed MQTT broker can issue motion commands, as the WebSocket bridge does.
- **H3 (map-artifact reuse).** Reusing the vendor's own map annotations is more reliable than re-deriving goal coordi-

nates offline.

- **H4 (transport $\neq$ execution).** A transport acknowledgment does not imply that the chassis executed the command.

Contributions.

C1. A reproducible seven-phase reverse-engineering methodology for closed Android–ROS robots, organised around falsifiable hypotheses and a triangulation rule, together with the effort it required in practice (Sections IV and VI).

C2. A recovered command surface for a commercial reception platform and an edge-first architecture built on it, including a speech channel obtained through Android inter-process communication after every protocol-level channel had been eliminated (Section V).

C3. Field evidence that execution must be verified against state telemetry rather than transport acknowledgments, with the resulting design rule, quantified indicators, and an analysis of which findings should transfer to other closed platforms (Sections VI and VII).

All work reported here was carried out on hardware owned by our institution, for interoperability purposes, without redistributing vendor code or circumventing any licensing mechanism; Section VII details the ethical and legal framing.

II. Related Work

A. Re-engineering commercial robot platforms

The closest precedent is Kim *et al.* [1], who re-engineered the software architecture of a commercial home service robot. Their study follows much the same arc as ours, but one difference decides everything: they had the manufacturer's source code and its engineering team, while we start from a sealed product. Mühe *et al.* [2] recovered a proprietary industrial robot language, again working from artifacts the vendor had published. Both begin with more material than we do, and the methodology below exists mainly to make up for that shortfall.

B. Vendor lock-in and interoperability

The trade-off between proprietary and open platform strategies is long established [3], and service robotics reproduces it one product at a time. Glauser *et al.* [4] report healthcare deployments held back by outdated and restrictive vendor SDKs, and propose an interoperable programming interface as the remedy. Middleware bridges such as ROS–YARP [5] address the adjacent problem of connecting two interfaces that are both *known*. In every case the interface itself is taken for granted. The question we take up comes earlier: how does a laboratory obtain one at all, on a robot it has already bought?

C. Black-box access to ROS systems

ROSBRIDGE was designed to let non-ROS clients reach a ROS graph over WebSocket [6], [7], normally deployed by the integrator who controls that graph. DeMarinis *et al.* [8] inverted this perspective, scanning the Internet for exposed ROS systems and showing that sensors and actuators can be reached without any knowledge of the node graph; Dieber *et al.* [9] treat the same openness as an application-level security problem. We occupy a similar position with the sign reversed: we too arrive at a vendor-deployed bridge as an unknown client, but constructively, and on hardware we own.

TABLE I
Positioning relative to the nearest categories of work

Work	Primary emphasis	Distinction in this paper
Kim *et al.* [1]	Re-architecting a service robot	Vendor source and engineers available; here the product is sealed
Mühe *et al.* [2]	Proprietary robot language	Published artifacts as input; here no artifact is published
DeMarinis *et al.* [8]	Exposed ROS systems at scale	Same black-box stance, applied constructively to one owned unit
Glauser *et al.* [4]	Interoperable interface for social robots	Assumes a usable SDK; we recover one where none exists
Giese *et al.* [13]	Companion-app protocol recovery (IoT)	Transposed to a robot whose app drives a separate ROS computer
This work	**Non-destructive protocol reconstruction**	**Hypothesis-driven method; execution verified against state, not transport**

D. Protocol recovery from companion software

Extracting protocol specifications from observed traffic is a mature security technique [10], and Android analysis has focused on information flow within an application [11] or between applications [12]. The practice of reading a device's companion app to recover its wire protocol is established in the IoT community [13]. Hypothesis H1 imports that practice into robotics, where the companion application controls a second, physically separate computer.

E. Edge robotics and on-device interaction

Cloud robotics assumes a reliable path to remote compute [14]. We had the opposite constraint. The robot's own network guarantees no Internet route, so speech and recognition had to run on the head, which pushed us towards closed-vocabulary recognition [15], [16] and on-device face embeddings [17]. Language-model agents [18], [19] assume an open action API is already available. On a closed robot that API has to be recovered first, which is the work reported here.

Table I sets out the gap. Each neighbouring line of work starts once the interface is known; what we describe here is the step before that.

III. Platform and Access Model

The study platform is a mass-produced indoor reception robot, sold as a turnkey product with a companion Android application and a subscription cloud service. It pairs a Linux/ROS chassis [20] carrying SLAM, a planner, LiDAR and RGB-D sensing with an Android 7.1 head that drives a 15.6 inch touchscreen, cameras and speakers. We refer to it generically throughout, since nothing in the methodology depends on the particular model and we have no interest in singling out one manufacturer. The two computers run independent network stacks, a fact no vendor document states and no application string reveals.

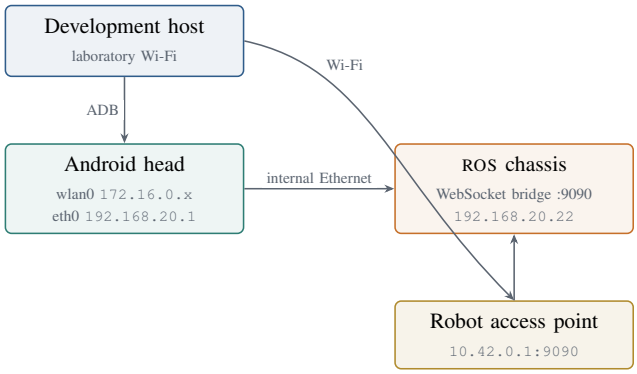

Fig. 2. Three network segments on a product marketed as a single device. Conflating the head's DHCP address with the chassis address was the most frequent operator error we observed.

Network mapping turned up three segments (Fig. 2), and that was already a result in itself. Two channels are reachable from outside the vendor stack: a WebSocket bridge on port 9090 and an MQTT broker on port 1883. Shell access to the chassis is closed. We observed that repeated authentication attempts trigger an account lockout, and went no further down that road, partly because it would break the non-destructive principle stated below and partly because we had one unit and bricking it would have ended the study. Two vendor packages sit on the head, a welcome application and a deployment tool, and both link against the same chassis-communication library. We treat that library as the *de facto* protocol specification. It is the empirical basis for H1.

IV. Methodology

A. Principles

Four rules govern the procedure.

- **P1 — observe before publishing.** We listen passively before injecting anything, because there is no way to predict how an undocumented safety-critical system will react.
- **P2 — verify effects, not acknowledgments.** A command counts as validated only once an independent state topic changes. This is the working form of H4.
- **P3 — triangulate.** We accept a finding only when two independent sources agree, drawn from network traces, introspection, decompiled application code and field logs.
- **P4 — remain non-destructive.** No firmware modification, no root, no persistent change to vendor partitions. This is an ethical commitment, and with a single unit it is also what makes the work repeatable.

B. Vantage points and their blind spots

The whole methodology exists because no single observation point tells us enough (Fig. 3). Introspection lists names, but says nothing about preconditions. Traffic capture shows what the vendor application happened to send during one session, which is not the same as what the chassis would accept. Decompiled code gives intent and the implicit setup steps, but leaves open whether a given path is reachable on this firmware. Triangulating between the three is not a stylistic preference; it is the only way to cover what any one of them misses.

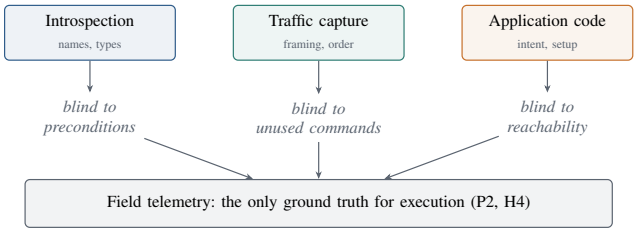

Fig. 3. Each vantage point answers a different question and is blind to the others. A recovered command is accepted only after state telemetry confirms a physical effect.

TABLE II
Reverse-engineering pipeline

#	Phase	Evidence produced	Hyp.	Stopping criterion
1	Network mapping	Segment map, open ports	-	All hosts and reachable ports enumerated
2	Introspection	Topic and service inventory	H1	Inventory stable across sessions
3	Passive broker	Payload classes on the broker	H2	No command payload after repeated sessions
4	Application audit	Names, types, implicit setup steps	H1	Every observed name located in code
5	Frame capture	Message framing and ordering	H1	Client reproduces vendor framing
6	Edge integration	On-device services and bridges	-	Capability reachable without a host PC
7	Field validation	State transitions, pose deltas	H3, H4	Effect confirmed on independent topic

C. The seven-phase pipeline

Table II gives, for each phase, the evidence it produces, the hypothesis it bears on, and the condition under which we stopped. The stopping criteria are not a formality. With 455 topics on the wire, an exploration without them simply does not converge.

We kept the negative results instead of discarding them: a legacy velocity topic with no subscriber, two candidate speech topics, three HTTP ports on the head, and the broker command path of H2. Each one closed a branch of the search, which is worth as much as an open one.

We report the cost of the method because a procedure nobody can budget for is of little use to anyone else. Phases 1–6 took two weeks and phase 7 a further week, spread over eight sessions with the physical robot and carried out by one full-time engineer. We decompiled two applications; introspection returned 455 topics and 308 services, a count taken from session logs rather than from an archived inventory. The exploration scripts, the recovered interface description and the logs behind the navigation, guided-tour, speech-bridge and question-matching results are released with this paper [21]. The remaining indicators in Table III are readings noted during field campaigns for which we kept no machine-readable log,

and we mark them as such rather than implying an archive that does not exist.

V. Recovered Command Surface and Edge Platform

A. Command and telemetry

Motion is commanded by advertising a multiplexer input and then publishing twist messages on it at roughly 10 Hz. The vendor application quietly sets a manual-control mode when a session opens. Our early builds did not, and every command we sent was dropped without comment. That precondition was visible in the decompiled application and in no traffic capture we took, and it was our first concrete encounter with H4. Autonomous motion comes in two forms: a coordinate goal, and a named-annotation service backed by the vendor's marker manager. One status field reports the state machine gating both, with values for unlocalized, ready, navigating, arrived and error. We throttle subscriptions per topic to keep the bridge usable over the robot's Wi-Fi.

One detail cost us time and is worth passing on. A single vendor message type, carrying one integer field, is reused across several unrelated services, each giving that integer its own private meaning. Call one of them with empty or guessed arguments and it returns success, then either does nothing or does the opposite of what was intended. We recovered the correct arguments through the same introspection route that supports H1. Interfaces reconstructed from the outside are prone to this, and an acknowledgment offers no protection against it.

B. Edge architecture

The reconstructed platform is edge-first by necessity, since the robot's network offers no guaranteed Internet path. A domain layer hides the robot behind one interface with two implementations, simulated and real. This let us exercise the logic under 169 unit tests without the physical machine, and it kept ordinary software defects separate from genuine protocol uncertainty, which during field sessions was the difference between a productive afternoon and a wasted one. Above it, an application layer exposes REST and WebSocket endpoints and runs either on a development machine or on the Android head, so a visitor session needs no external computer. A presentation layer serves the visitor interface of Fig. 1 and the operator console.

C. Speech through inter-process communication

Speech was the hardest capability to recover, and at the protocol level the search simply failed. No topic accepted text, no such service existed, the head's HTTP ports were closed, and the broker carried telemetry only. The reason turned out to be architectural rather than accidental. The vendor implements speech inside the Android application layer, over opaque IPC to a bundled engine, so it was never a robot-level capability at all and no amount of protocol search would have found it. The answer was to stop looking sideways and move up a layer. A 50 kB companion application registers a broadcast receiver [12] and hands incoming text to the platform's native speech engine:

```
am broadcast -n com.cybel.ttsbridge/.SpeakReceiver
  -a com.cybel.ttsbridge.SPEAK --es text "Welcome."
```

The move itself generalises. Whenever a vendor implements a function in the application layer, no protocol-level search will reach it, and the host operating system is where the capability has to be picked up instead.

D. Interaction layer

Visitor interaction runs entirely on the head. We abandoned free dictation early: a compact offline model holds no pronunciation for the proper nouns that matter on a campus, and it transcribed them unreliably in front of visitors. Recognition now works from a closed vocabulary rebuilt at request time out of the deployed annotations and the question set. The same limitation caught us again with the wake phrase. Built from the platform's invented name, it was consistently decoded as an unrelated word the model did in fact know; replacing it with a phonetically close phrase made of real words solved the problem. Face re-identification runs on the head at roughly 1 Hz and sends only embedding vectors, never images.

VI. Experimental Evaluation

Our samples are small, so we give proportions with Wilson score intervals [22], [23], which stay well behaved at these sizes, and we state every trial count.

A. H1 confirmed, H2 rejected

Every topic and service name we later exercised on the wire had first been found in the decompiled vendor library, and so had preconditions like the manual-mode gate that no traffic capture ever showed us. There is a structural reason for this. The vendor's own application has to speak the real protocol in order to drive the chassis, so its compiled code is a lower bound on the command surface in a way that documentation, which may describe intentions rather than behaviour, is not. We did not test the converse. Whether commands exist that the application never uses remains open, and Section VII records it as a limitation.

H2 is rejected. Wildcard subscription across several sessions returned one telemetry topic carrying odometry, and no payload we replayed changed the robot's state. The vendor library explains the result: its broker client is wired only to telemetry publishers, and the command path runs exclusively over the WebSocket bridge. On this platform the broker cannot carry commands at all. We report the rejection because MQTT is a reasonable first guess for anyone approaching a machine like this, and rediscovering that it leads nowhere is expensive.

B. H4: transport acknowledgment is not execution

The bridge acknowledges any syntactically valid message it can route to a subscriber. That acknowledgment is produced above the chassis safety and mode-arbitration layer, which means it certifies the first hop and nothing beyond it (Fig. 4). Our field record shows what this costs in practice. Across

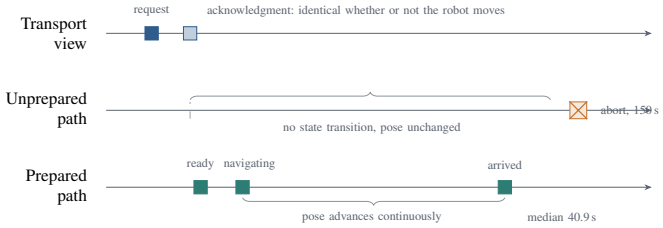

Fig. 4. The transport lane is identical whether or not the command executes. Only an independent state topic distinguishes the two lower cases. Timings from the bench comparison of Section VI-C ($n = 3$ per arm).

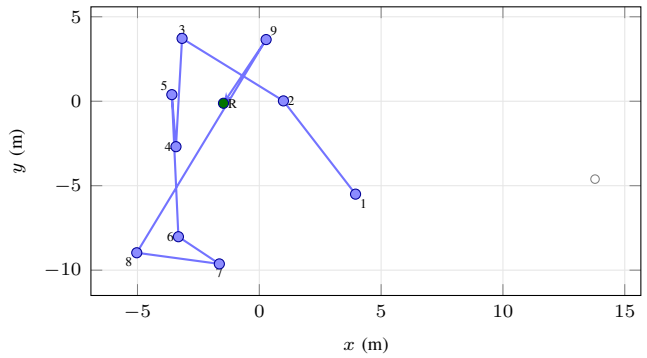

Fig. 5. Guided-tour route over the annotations deployed through the vendor application, in visiting order: 9 resolvable stops and a return leg (R), 10 navigation legs per tour. Hollow markers are annotations that exist on the map but are not on the tour. Stops: 1 PORTE-LABO; 2 CNC ROUTEUR; 3 IMPRIMANTE 3D; 4 POINT-MACHINE; 5 THERMOFORMAGE; 6 EXTRUSION-SOUFFLAGE; 7 POSTE-REMPLISSAGE-BOUCHONNAGE; 8 POSTE-ETIQUETAGE; 9 SÉRIGRAPHIE; R POINT-RECHARGE. Coordinates are the values the marker manager reports; reusing the annotations avoids the drift that affects coordinates extracted offline, but does not by itself make navigation succeed (Section VI-C).

17 logged navigation commands, the outcome followed the reported localization state exactly: all ten failures happened while the robot reported itself unlocalized, all seven successes while it reported navigating. The same split holds for both goal-addressing schemes, coordinate goals failing five times and succeeding once, named annotations failing five times and succeeding six. In this record the addressing scheme explains nothing and the precondition state explains everything. Every one of the failed commands had been acknowledged.

C. H3: addressing scheme versus preparation

Goal annotations are created in the vendor deployment application (Fig. 5) and resolved by the chassis marker manager. We compared the two schemes directly on the bench, with automatic mode set and telemetry subscribed for both arms, navigating to the same annotated target (Fig. 6 (a)). Coordinate goals succeeded in 3 of 3 trials, reaching the arrived state in 40.0 s, 40.9 s and 46.5 s (median 40.9 s). The annotation service, issued from the same bare client through its full fallback chain, produced no state transition at all in 3 of 3 trials and was abandoned at the 150 s timeout. At three trials per arm the difference is not statistically significant (Fisher's exact test, two-sided, $p = 0.10$), and neither arm's interval excludes the other's point estimate.

Taken alone this would be a clean verdict against annotation reuse, and it would also contradict everything we saw in daily operation, where the annotation route is precisely the one that works. Three consecutive guided tours ran to completion without operator intervention, 30 navigation legs in total counting nine resolvable stops and a return leg per tour, with durations of 667.6 s, 670.4 s and 822.3 s. What reconciles the two observations is everything the bare client leaves out. The operational path checks control state, clears blocking navigation states, relocalizes when the robot reports itself unlocalized or below the 60 % confidence gate, and declines to issue a goal if those conditions are not met. The bare client does none of this and fires into the void.

We therefore do not claim H3 in the form we stated it. What the evidence supports is broader and more useful: what decides the outcome is not how the goal is addressed, but whether the preconditions the vendor application sets implicitly have been re-established explicitly. Annotation reuse is still the right operational choice, because it avoids the drift between offline coordinates and the live map, but its reliability belongs to the preparation sequence rather than to the addressing scheme. Two explanations remain consistent with the bench failure, an unmet precondition or a target name the marker manager did not resolve in that session, and we could not separate them with the instrumentation we had.

D. System indicators

Table III collects the measured indicators. We took none of them for their own sake, so the table states what each one is meant to settle; an indicator that settles nothing is padding, and we have left several such figures out. Guided-tour completion appears at two levels, because the 30 legs are not independent observations: a leg that fails ends the rest of its tour. The leg-level interval is therefore optimistic, and the tour-level one is the conservative reading.

Two negative results deserve as much visibility as the positive ones. On a controlled set of 12 questions with four paraphrases each, question matching succeeded in 27 of 48 trials (56.2 %). Most of the failures were near misses, where overlapping keywords routed a question to a neighbouring entry, rather than cases where nothing matched at all. That pattern argues for a wider synonym set or a semantic fallback, not for moving the threshold. We also timed the complete voice loop, from the end of a visitor's utterance to the start of the spoken reply, because it is what decides whether an exchange feels like a conversation. Over 12 campaign readings it ranged from 979 ms to 23 s. The distribution is badly skewed and we quote no mean, which here would describe none of the observations.

E. Comparison with the vendor application

Table IV compares the two systems capability by capability. It is not a controlled user study, and where both provide a function we make no claim about which does it better. The

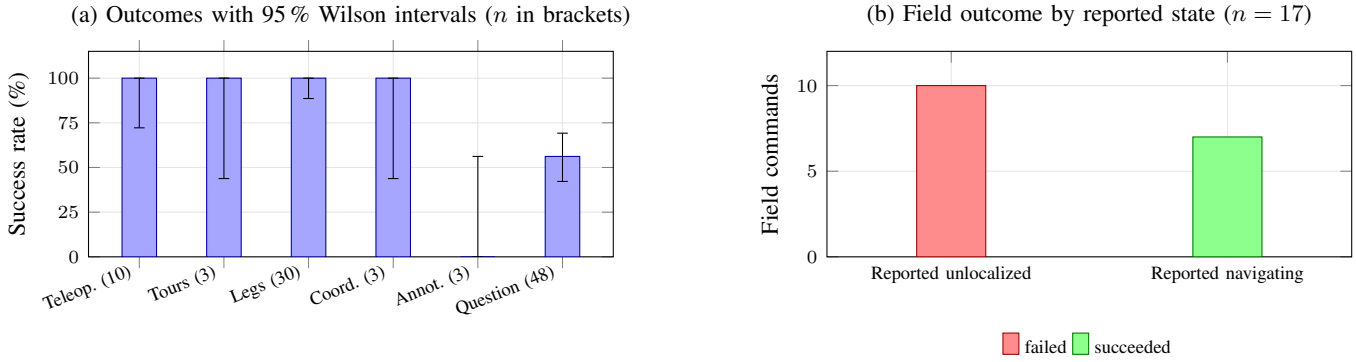

Fig. 6. (a) Measured outcomes. Intervals are wide because samples are small; the annotation arm of the bench comparison and the coordinate arm do not separate at $p = 0.10$. (b) In the field record, reported localization state separates success from failure perfectly, and the addressing scheme does not separate them at all.

TABLE III
MEASURED INDICATORS, JUNE–JULY 2026, AND WHAT EACH ONE IS FOR. LEGS WITHIN A TOUR ARE NOT INDEPENDENT OBSERVATIONS, SO THE LEG-LEVEL INTERVAL IS OPTIMISTIC. ROWS MARKED † ARE CAMPAIGN READINGS FOR WHICH NO MACHINE-READABLE LOG WAS ARCHIVED.

Indicator	n	**Value**	**95 % interval**	**What it establishes**
Teleoperation success†	10	100 %	72.2–100 %	That the manual-mode precondition recovered from the vendor application (H1) is the correct one: without it the same commands were dropped silently
Guided tours completed	3	100 %	43.8–100 %	That the full preparation path works end to end in service, unattended
individual legs	30	100 %	88.6–100 %	The same at leg resolution, which is the finest grain the tour logs support
Tour duration (median)	3	670 s	668–822 s	The operating envelope of a complete visitor session, for anyone sizing a deployment
Coordinate goal, bench	3	100 %	43.8–100 %	Arm A of the H3 comparison: goals addressed by coordinate, preparation omitted
Annotation goal, bare client	3	0 %	0–56.2 %	Arm B: same target and same conditions, and the measurement that redirected H3 towards H4
Repeat-question match	48	56.2 %	42.3–69.3 %	The ceiling of the closed-vocabulary matcher, and the main negative result of the study
Speech-bridge latency (mean)	5	923 ms	651–1555 ms	The price of routing speech through the host operating system instead of the vendor engine
Local REST latency†	–	< 100 ms	–	That the reconstructed stack adds no delay a visitor would notice on top of the bridge
Wi-Fi round-trip time†	–	–	89–1654 ms	Why telemetry is throttled per topic, and why the round trips removed in Section VII-A were worth removing

TABLE IV
CAPABILITY COMPARISON WITH THE VENDOR APPLICATION

Capability	**Vendor**	**This work**
Teleoperation, guided tour	yes	yes
Speech synthesis	yes	yes (IPC bridge)
Speech recognition	yes (cloud)	yes (on-device)
Face re-identification	yes	yes (on-device)
Question answering	yes (cloud)	yes (local)
Operates without Internet	no	yes
Inspectable and modifiable logic	no	yes
Self-hostable, no vendor account	no	yes
Multi-floor navigation	yes	no
Fleet management console	yes	no
Maintenance and updates under warranty	yes	no
Recognition quality across languages	yes	partial

lower block lists the areas where the vendor keeps a real advantage, most of which follow from resources a laboratory does not have.

VII. DISCUSSION

A. What the study establishes

The methodological lesson is that discovery and verification are two different problems, and on a closed platform the second is much the harder. Recovering names is fairly mechanical: introspection plus a decompiler produce a large candidate surface in a matter of days. Establishing that any of those commands does something is where the time actually goes, because the interface reports success in exactly the same way whether or not the machine moves. Our field record puts this as starkly as we could have wished. The outcome was settled by a state field, and not at all by the command variant we had spent weeks trying to tell apart.

From this follows a design rule, and the rule has a price. Pair every command with an independent observation of its effect, and re-establish explicitly every precondition the vendor application sets implicitly. That preparation sequence is what separates a working integration from a bench script that reports

success and moves nothing. It is also not free. We had a standing complaint that the robot felt sluggish, and traced it to preparation steps being issued unconditionally on every command, which added two round trips and a fixed delay even when telemetry already confirmed the required state. Skipping them once the state is confirmed is the same discipline read backwards: do not re-issue a command whose effect the telemetry is already reporting.

B. Generalization

H1 should hold wherever the vendor's companion application is the primary control surface, since that application has no choice but to speak the true protocol. H4 should hold on any platform that places a safety or mode-arbitration layer between the message bus and the actuators, which is the norm for commercial mobile bases. The rejection of H2 is specific to this manufacturer, and another may well command over a broker; what carries across is the triangulation procedure that settled the question in a single audit instead of weeks of intermittent failures. Concrete names, gate semantics and naming conventions do not transfer at all, and have to be rediscovered, which is what the pipeline is for. We have run the method on one platform. Whether it transfers to a second vendor is a claim we would like to make and cannot yet support.

C. Ethics and data protection

The robot belongs to our institution and was studied for interoperability. We analysed the vendor application in order to recover message schemas, not to extract or redistribute proprietary assets, and we bypassed no licensing or protection mechanism. Staying non-destructive was an ethical commitment and a practical one at the same time: it leaves the warranty and the update path intact, and it avoids producing a firmware fork that nobody would maintain.

Face re-identification processes biometric data, and that needed a separate judgement. Pretrained embedding models differ in the provenance of their training data, and several widely circulated conversions descend from a corpus that was withdrawn over consent concerns. We chose a model whose provenance is documented, obtained through a checksum-verified chain, and we record the trade-off here rather than presenting the choice as settled. Enrollment is never automatic. A staff member initiates it in the presence of the person being enrolled, who is told at that moment what is happening, and individual embeddings can be deleted afterwards from the operator interface. Only embedding vectors leave the head. No image is stored or transmitted.

D. Limitations

The evaluation is where this work is weakest. Our samples are small and the intervals in Table III are wide to match; the bench comparison in particular cannot support a claim of difference at $n = 3$ per arm, and we do not make one. We never instrumented session-level connection reliability, so we describe the reconnection mechanism instead of quoting a figure we cannot stand behind. The vendor comparison covers capabilities, not measured task performance. H1 was confirmed in one direction only: observed names do appear in the application, but we did not establish that the application exercises every command the firmware will accept, so the surface we recovered is a lower bound. Ten recorded wake events are too few to estimate a false-trigger rate, and the face-matching threshold has not been calibrated for several visitors enrolled at once. All of it comes from one robot, one map and one site.

VIII. Conclusion

We asked whether the command surface of a closed Android–ROS service robot can be recovered without the manufacturer's help, and what evidence that recovery demands. The seven-phase, triangulation-based pipeline produced a working surface on a commercial reception platform, and carried an edge deployment that replaces the vendor cloud for teleoperation, navigation, speech and visitor interaction. Static analysis of the vendor application proved a dependable source of protocol names and of the preconditions nobody documents (H1). The exposed broker turned out to be incapable of carrying commands at all (H2). Above everything else, the gap between transport and execution (H4) governed the entire exercise: the addressing scheme we had expected to matter (H3) explained none of our field outcomes, while the precondition state explained all of them.

The obvious next steps are to run the method against a second manufacturer, to replace our capability table with a measured task comparison, to collect enough navigation trials for the two arms to separate, and to ground a language-model planner in the primitives we validated rather than leave it to rediscover the interface for itself.

References

[1] M. Kim, J. Lee, K. C. Kang, Y. Hong, and S. Bang, "Re-engineering software architecture of home service robots: A case study," in *Proc. 27th International Conference on Software Engineering (ICSE)*, 2005, pp. 505–513.

[2] H. Mühe, A. Angerer, A. Hoffmann, and W. Reif, "On reverse-engineering the KUKA Robot Language," in *1st International Workshop on Domain-Specific Languages and Models for Robotic Systems (DSLRob)*, 2010, arXiv:1009.5004.

[3] J. West, "How open is open enough? Melding proprietary and open source platform strategies," *Research Policy*, vol. 32, no. 7, pp. 1259–1285, 2003.

[4] R. Glauser, J. Holm, M. Bender, and T. Bürkle, "How can social robot use cases in healthcare be pushed with an interoperable programming interface," *BMC Medical Informatics and Decision Making*, vol. 23, no. 1, p. 118, 2023.

[5] M. Aragão, P. Moreno, and A. Bernardino, "Middleware interoperability for robotics: A ROS–YARP framework," *Frontiers in Robotics and AI*, vol. 3, p. 64, 2016.

[6] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "Rosbridge: ROS for non-ROS users," in *Proc. 15th International Symposium on Robotics Research (ISRR)*, 2012.

[7] Open Source Robotics Foundation, "rosbridge_suite," http://wiki.ros.org/rosbridge_suite, ROS Wiki, accessed 2026.

[8] N. DeMarinis, S. Tellex, V. P. Kemerlis, G. Konidaris, and R. Fonseca, "Scanning the Internet for ROS: A view of security in robotics research," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8514–8521.

[9] B. Dieber, S. Kacianka, S. Rass, and P. Schartner, "Application-level security for ROS-based applications," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 4477–4482.

[10] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, "Prospex: Protocol specification extraction," in *Proc. IEEE Symposium on Security and Privacy*, 2009, pp. 110–125.

[11] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," *ACM Transactions on Computer Systems*, vol. 32, no. 2, pp. 1–29, 2014.

[12] E. Chin, A. P. Felt, K. Greenwood, and D. Wagner, "Analyzing inter-application communication in Android," in *Proc. 9th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2011, pp. 239–252.

[13] D. Giese and D. Wegemer, "Having fun with IoT: Reverse engineering and hacking of Xiaomi IoT devices," DEF CON 26, 2018.

[14] B. Kehoe, S. Patil, P. Abbeel, and K. Goldberg, "A survey of research on cloud robotics and automation," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 398–409, 2015.

[15] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, arXiv:1804.03209.

[16] Alpha Cephei, "Vosk: Offline open-source speech recognition toolkit," https://alphacephei.com/vosk/, accessed 2026.

[17] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.

[18] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. 6th Conference on Robot Learning (CoRL)*, 2022.

[19] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter *et al.*, "PaLM-E: An embodied multimodal language model," in *Proc. 40th International Conference on Machine Learning (ICML)*, 2023.

[20] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, 2009.

[21] Anonymous Authors, "Reverse-engineering scripts, edge platform and field logs," https://anonymous.4open.science/r/ANONYMOUS-REPO-ID, 2026, anonymised for review.

[22] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[23] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statistical Science*, vol. 16, no. 2, pp. 101–133, 2001.